UNIT-4

TOPIC-1 NUMPY:

1Q) What is numpy?

Ans:

NumPy is a Python library used to handle large multi-dimensional arrays and matrices efficiently.

2Q) What are the applications of Numpy?

Ans:

NumPy is widely used in **data science**, **machine learning**, **scientific computing**, and **engineering** applications due to its speed, versatility, and ease of use.

3Q) Why use Numpy?

Ans:

Python's built-in lists and loops are slow for large data. NumPy provides:

- **Better performance**
- **Cleaner syntax**

- **Powerful operations** on entire datasets without writing explicit loops

It forms the **foundation** for many other libraries:

- **Pandas**: Data analysis
- **SciPy**: Scientific computing
- **Scikit-learn**: Machine learning
- **TensorFlow / PyTorch**: Deep learning

4Q) What are features of Numpy?

Ans:

Feature	Simple Explanation
1. nd array (N-dimensional array)	It is the main data structure in NumPy. It stores numbers in the form of arrays and works much faster than Python lists. It supports 1D, 2D, 3D, and higher-dimensional arrays.
2. Broadcasting	It allows us to perform operations on arrays of different sizes without writing extra code or loops.
3. Vectorized Operations	It lets us perform calculations on all elements of an array at once, without using for loops, making the code faster and simpler.

4. Mathematical Functions	NumPy provides many built-in functions like sum(), mean(), max(), min(), sqrt(), and log() to perform mathematical calculations easily.
5. Linear Algebra Support	NumPy provides functions to perform matrix operations such as matrix multiplication, finding determinants, inverses, and solving equations.
6. Random Number Generation	NumPy can generate random numbers and random data, which are useful for testing programs, simulations, and machine learning.

5Q) What are the advantages of Numpy?

Ans:

- **Speed**: NumPy arrays are implemented in C, which makes them significantly faster than native Python lists.
- **Memory Efficient**: Requires less memory and storage for the same data.
- **Convenient Syntax**: Supports slicing, masking, filtering, and reshaping easily.
- **Interoperability**: Can integrate with C, C++, Fortran, and other Python libraries.
- **Extensibility**: Forms the foundation for advanced libraries in AI/ML.

6Q) What are the applications of Numpy?

Ans:

- **Data Analysis:** NumPy is the core of Pandas, which is used for cleaning and manipulating large datasets.
- **Machine Learning:** Libraries like Scikit-learn, TensorFlow use NumPy internally for data manipulation.
- **Image Processing:** Images are stored as NumPy arrays; useful in OpenCV, PIL.
- **Scientific Simulations:** Physics, chemistry, and engineering simulations use NumPy arrays for computation.
- **Finance:** Analyzing stock data, calculating returns, moving averages, etc.

7Q) Write differences between List and Numpy.

Ans:

List	Array
List can have elements of different data types for example, [1,3.4, 'hello', 'a@']	All elements of an array are of same data type for example, an array of floats may be: [1.2, 5.4, 2.7]
Elements of a list are not stored contiguously in memory.	Array elements are stored in contiguous memory locations. This makes operations on arrays faster than lists.
Lists do not support element wise operations, for example, addition, multiplication, etc. because elements may not be of same type.	Arrays support element wise operations. For example, if A1 is an array, it is possible to say A1/3 to divide each element of the array by 3.
Lists can contain objects of different datatype that Python must store the type information for every element along with its element value. Thus lists take more space in memory and are less efficient.	NumPy array takes up less space in memory as compared to a list because arrays do not require to store datatype of each element separately.
List is a part of core Python.	Array (ndarray) is a part of NumPy library.

8Q) Explain creation of 1D, 2D and 3D arrays in numpy .

Ans:

Creating 1D, 2D, and 3D Arrays in NumPy

NumPy arrays are created using the `np.array()` function.

import numpy as np

1. One-Dimensional (1D) Array

A **1D array** is a simple list of elements arranged in a single row.

Syntax

arr = np.array([10, 20, 30, 40, 50])

Example

import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr)

Output

[10 20 30 40 50]

Shape

print(arr.shape)

Output

(5,)

Explanation

- There are **5 elements**.
- Since it has only one dimension, the shape is **(5,)**.
- The comma indicates that it is a **1D array**.

2. Two-Dimensional (2D) Array

A **2D array** consists of **rows and columns**, just like a table or matrix.

Syntax

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
```

Example

```
import numpy as np


arr = np.array([[1, 2, 3],
                [4, 5, 6]])


print(arr)
```

Output

```
[[1 2 3]
 [4 5 6]]
```

Shape

```
print(arr.shape)
```

Output

```
(2, 3)
```

Explanation

- **2 rows**
- **3 columns**
- Shape is **(2, 3)**.

3. Three-Dimensional (3D) Array

A **3D array** is a collection of multiple 2D arrays.

Syntax

```
arr = np.array([
   [[1,2,3],
    [4,5,6]],

   [[7,8,9],
    [10,11,12]]
])
```

Example

```
import numpy as np


arr = np.array([
   [[1,2,3],
    [4,5,6]],


   [[7,8,9],
    [10,11,12]]
])


print(arr)
```

Output

```
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
```

Shape

```
print(arr.shape)
```

Output

(2, 2, 3)

Explanation

- **2** → Number of 2D arrays (depth/layers)
- **2** → Number of rows in each 2D array
- **3** → Number of columns in each row

So, the shape is **(2, 2, 3)**.

Understanding `shape`

The **shape** of an array tells us the **size of the array in each dimension**.

Array Type	Example Shape	Meaning
1D	`(5,)`	5 elements
2D	`(2,3)`	2 rows, 3 columns
3D	`(2,2,3)`	2 layers, 2 rows, 3 columns

Visual Representation

1D Array → Shape `(5,)`

[10 20 30 40 50]

- 5 elements

2D Array → Shape `(2,3)`

[

[1 2 3]

[4 5 6]

]

- 2 rows
- 3 columns

3D Array → Shape `(2,2,3)`

Layer 1

[

[1 2 3]

[4 5 6]

]

Layer 2

[

[7 8 9]

[10 11 12]

]

- 2 layers (2D arrays)

- Each layer has 2 rows
- Each row has 3 columns

Exam Tip

Formula to remember:

- **1D Shape** → `(Number of elements,)`
- **2D Shape** → `(Rows, Columns)`
- **3D Shape** → `(Layers, Rows, Columns)`

10Q) Write Numpy attributes.

Ans:

1. ndim → Number of Dimensions

Tells how many dimensions (or axes) the array has.

- A **1D array** is a line of elements.
- A **2D array** is a table (rows × columns).
- A **3D array** is like a cube or a stack of 2D tables.

Example:

import numpy as np

a = np.array([1, 2, 3]) # 1D array

print(a.ndim) # Output: 1

b = np.array([[1, 2], [3, 4]]) # 2D array

print(b.ndim) # Output: 2

2. shape → Shape of the Array

Tells how many elements are in each dimension.
It returns a tuple: (rows, columns) for 2D, etc.

Example:

b = np.array([[1, 2], [3, 4]])

print(b.shape) # Output: (2, 2) → 2 rows, 2 columns

Think of it like:

[[1, 2],

[3, 4]]

3. size → Total Number of Elements

Total count of all elements in the array.

Example:

b = np.array([[1, 2], [3, 4]])

print(b.size) # Output: 4 → (2 × 2)

4. dtype → Data Type of Array Elements

Shows what type of numbers are stored in the array:
e.g., int32, float64, bool, etc.

Example:

```
a = np.array([1, 2, 3])

print(a.dtype)   # Output: int64

b = np.array([1.5, 2.5])

print(b.dtype)   # Output: float64
```

11Q) Write Numpy array functions.

Ans:

1. np.zeros(shape)

Definition:

Creates an array filled with **zeros**.

Example:

```
import numpy as np

a = np.zeros((2, 3))

print(a)
```

Explanation:

- np.zeros((2, 3)): creates a 2x3 matrix of 0s.
- a stores the array.
- print(a): prints the result.

Output:

[[0. 0. 0.]

[0. 0. 0.]]

2. np.ones(shape)

Definition:

Creates an array filled with **ones**.

Example:

a = np.ones((3, 2))

print(a)

Explanation:

- (3, 2) → 3 rows, 2 columns.
- All values are 1.

Output:

[[1. 1.]

[1. 1.]

[1. 1.]]

3. np.full(shape, fill_value)

Definition:

Creates an array filled with a **specific value**.

Example:

a = np.full((2, 3), 7)

print(a)

Explanation:

- (2, 3) is shape.
- 7 is the fill value.

Output:

[[7 7 7]

[7 7 7]]

4. np.linspace(start, stop, num)

Definition:

Generates **evenly spaced values** from start to stop.

Example:

a = np.linspace(0, 1, 5)

print(a)

Explanation:

- Start = 0, Stop = 1.
- Generates 5 values **evenly spaced.**

Output:

[0. 0.25 0.5 0.75 1.]

5. np.arange(start, stop, step)

Definition:

Creates numbers from start to stop (exclusive), with a step.

Example:

a = np.arange(1, 10, 2)

print(a)

Explanation:

- Starts at 1, ends before 10, step size = 2.

Output:

[1 3 5 7 9]

6. np.sort(array)

Definition:

Returns a **sorted copy** of an array.

Example:

a = np.array([4, 2, 8, 1])

print(np.sort(a))

Explanation:

- np.sort() returns a new sorted array (does not change the original).

Output:

[1 2 4 8]

7. np.identity(n)

Definition:

Creates an n x n **identity matrix** (1s on diagonal).

Example:

a = np.identity(3)

print(a)

Explanation:

- Diagonal is all 1s, others 0.

Output:

[[1. 0. 0.]

[0. 1. 0.]

[0. 0. 1.]]

8. np.random module

Definition:

Used to generate **random numbers**.

Example: np.random.rand()

a = np.random.rand(2, 2)

print(a)

Explanation:

- Generates random float numbers in [0, 1).
- Shape is (2, 2).

Output: *(varies every time)*

[[0.45 0.89]

[0.67 0.12]]

9. np.random.shuffle(array)

Definition:

Shuffles the array **in place** along the first axis.

Example:

a = np.array([1, 2, 3, 4, 5])

np.random.shuffle(a)

print(a)

Explanation:

- shuffle() randomly reorders elements of a.

Output:

[3 1 4 2 5] # Example; changes each time

10. np.unique(array)

Definition:

Returns **sorted unique elements** from the array.

Example:

a = np.array([1, 2, 2, 3, 3, 3])

print(np.unique(a))

Explanation:

- Removes duplicates and sorts.

Output: [1 2 3]

12Q) Write Numpy Aggregations and Statistical Functions.

Ans:

1. np.sum(array)

Definition:

Returns the sum of all elements in the array.

Example:

import numpy as np

arr = np.array([1, 2, 3, 4])

print(np.sum(arr))

Explanation:

- np.array([1, 2, 3, 4]) creates a 1D array.
- np.sum(arr) adds all elements: 1+2+3+4 = 10.

Output: 10

2. np.mean(array)

Definition:

Returns the average (mean) of all elements.

Example:

arr = np.array([1, 2, 3, 4])

print(np.mean(arr))

Explanation:

- Mean = (1+2+3+4)/4 = 2.5.

Output: 2.5

3. np.std(array)

Definition:

Returns the standard deviation of the array elements.

Example:

arr = np.array([1, 2, 3, 4])

print(np.std(arr))

Explanation:

- Measures how much values deviate from the mean.
- Lower std → values are close to the mean.

Output: ≈ 1.118

4. np.median(array)

Definition:

Returns the median (middle value) of the array.

Example:

arr = np.array([1, 3, 2, 4])

print(np.median(arr))

Explanation:

- Sorted: [1, 2, 3, 4]
- Median = average of middle two = (2+3)/2 = 2.5

Output: 2.5

5. np.var(array)

Definition:

Returns the variance – the average of squared differences from the mean.

Example:

arr = np.array([1, 2, 3, 4])

print(np.var(arr))

Explanation:

- Mean = 2.5

- Variance = average of $[(1-2.5)^2, (2-2.5)^2, ...]$
- $= 1.25$

Output: 1.25

6. np.min(array)

Definition:

Returns the smallest value in the array.

Example:

arr = np.array([3, 7, 2, 9])

print(np.min(arr))

Output: 2

7. np.max(array)

Definition:

Returns the largest value in the array.

Example:

arr = np.array([3, 7, 2, 9])

print(np.max(arr))

Output: 9

Function Name	NaN-safe Version	Description
np.sum	np.nansum	Compute sum of elements
np.prod	np.nanprod	Compute product of elements
np.mean	np.nanmean	Compute median of elements
np.std	np.nanstd	Compute standard deviation
np.var	np.nanvar	Compute variance
np.min	np.nanmin	Find minimum value
np.max	np.nanmax	Find maximum value
np.argmin	np.nanargmin	Find index of minimum value
np.argmax	np.nanargmax	Find index of maximum value
np.median	np.nanmedian	Compute median of elements
np.percentile	np.nanpercentile	Compute rank-based statistics of elements
np.any	N/A	Evaluate whether any elements are true
np.all	N/A	Evaluate whether all elements are true

13Q) Write Numpy Element Wise Arithmetic and comparison operations.

Ans:

1. Element-wise Arithmetic Operations

a.np.add(array1, array2)

Definition: Adds corresponding elements.

Example:

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.add(a, b)
print(result)
```

Explanation:

- [1+4, 2+5, 3+6] → [5, 7, 9]

Output: [5 7 9]

b. np.subtract(array1, array2)

Definition: Subtracts elements of array2 from array1.

```
result = np.subtract(a, b)
print(result)
```

Output: [-3 -3 -3]

c. np.multiply(array1, array2)

Definition: Multiplies corresponding elements.

```
result = np.multiply(a, b)
```

print(result)

Output: [4 10 18]

d.np.divide(array1, array2)

Definition: Divides elements of array1 by array2.

result = np.divide(a, b)

print(result)

Output: [0.25 0.4 0.5]

e. np.mod(array1, array2)

Definition: Computes the remainder (modulo) element-wise.

a = np.array([10, 20, 30])

b = np.array([3, 7, 9])

result = np.mod(a, b)

print(result)

Output: [1 6 3]

Operator	Equivalent ufunc	Description
+	np.add()	Addition (e.g., 1 + 1 = 2)
-	np.subtract()	Subtraction (e.g., 3 - 2 = 1)
-	np.negative()	Unary negation (e.g., -2)
*	np.multiply()	Multiplication (e.g., 2 * 3 = 6)
/	np.divide()	Division (e.g., 3 / 2 = 1.5)
//	np.floor_divide()	Floor division (e.g., 3 // 2 = 1)
**	np.power()	Exponentiation (e.g., 2 ** 3 = 8)
%	np.mod()	Modulus/remainder (e.g., 9 % 4 = 1)

2. Element-wise Comparison Operations

<, >, <=, >=, ==, !=

Definition: Returns a boolean array comparing each element.

```
a = np.array([1, 2, 3])
b = np.array([2, 2, 2])
print(a < b)   # [True False False]
print(a >= b)  # [False True True]
```

Explanation:

Compares a[i] with b[i] for each i.

3. Rounding Functions

a. np.round(array, decimals)

Definition: Rounds elements to the given decimal places.

a = np.array([1.234, 5.678])

print(np.round(a, 1))

Output: [1.2 5.7]

b.np.floor(array)

Definition: Rounds down to the nearest integer.

a = np.array([1.9, 2.3])

print(np.floor(a))

Output: [1. 2.]

c.np.ceil(array)

Definition: Rounds up to nearest integer.

print(np.ceil(a))

Output: [2. 3.]

d.np.trunc(array)

Definition: Truncates (removes decimal part).

print(np.trunc(a))

Output: [1. 2.]

15Q) Write about indexing in Numpy.

Ans:

NumPy Indexing

What is Indexing?

Indexing is the process of accessing a specific element from a NumPy array using its position (index).

In NumPy, indexing starts from `0`, which means:

- **First element → Index `0`**
- **Second element → Index `1`**
- **Third element → Index `2`**
- **and so on.**

You can also use negative indexing to access elements from the end of the array.

- **Last element → Index `-1`**
- **Second last element → Index `-2`**

1. Indexing in a 1D Array

A 1D array has only one index.

Example

```
import numpy as np


arr = np.array([10, 20, 30, 40, 50])


print(arr[0])
print(arr[2])
print(arr[-1])
```

Output

```
10
30
50
```

Explanation

- `arr[0]` → First element → `10`
- `arr[2]` → Third element → `30`

- `arr[-1]` → Last element → `50`

2. Indexing in a 2D Array

A 2D array has rows and columns.

To access an element, use:

array[row_index][column_index]

or

array[row_index, column_index]

Example

```
import numpy as np

arr = np.array([[10, 20, 30],
                [40, 50, 60]])

print(arr[0][1])
print(arr[1][2])
```

```
# Another way
print(arr[0,1])
```

Output

```
20
60
20
```

Explanation

Array:

```
    Column
      0   1   2
Row 0  10  20  30
Row 1  40  50  60
```

- `arr[0][1]` → Row 0, Column 1 → `20`
- `arr[1][2]` → Row 1, Column 2 → `60`
- `arr[0,1]` is another way of writing `arr[0][1]`

3. Indexing in a 3D Array

A 3D array consists of multiple 2D arrays (layers).

To access an element, use:

array[layer][row][column]

or

array[layer, row, column]

Example

```
import numpy as np


arr = np.array([
    [[1,2,3],
     [4,5,6]],

    [[7,8,9],
     [10,11,12]]
])


print(arr[0][1][2])
print(arr[1][0][1])
```

Output

6

8

Explanation

Layer 0

[

[1 2 3]

[4 5 6]

]

Layer 1

[

[7 8 9]

[10 11 12]

]

- `arr[0][1][2]`
  - **Layer 0**
  - **Row 1**

    - Column 2
    - Value = 6
- `arr[1][0][1]`
    - Layer 1
    - Row 0
    - Column 1
    - Value = 8

Positive vs Negative Indexing

Example

```
arr = np.array([10,20,30,40,50])

print(arr[0])

print(arr[-1])

print(arr[-2])
```

Output

```
10

50

40
```

Explanation

Index	Value

0	10
1	20
2	30
3	40
4	50
-1	50
-2	40
-3	30

Quick Revision

Array Type	Indexing Syntax	Example
1D	`arr[index]`	`arr[2]`
2D	`arr[row][column]` or `arr[row, column]`	`arr[1][2]`

3D	`arr[layer][row][column]` `arr[layer, row, column]`	**or** `arr[0][1][2]`

16Q) Write about Slicing in Numpy.

Ans:

Slicing allows you to extract parts of a sequence (like lists, strings, or NumPy arrays) using this syntax:

sequence[start:stop:step]

Breakdown:

- start → index to begin from (inclusive)
- stop → index to stop at (exclusive)
- step → how many steps to skip

If you leave any of these blank, Python uses defaults:

- start = 0
- stop = len(sequence)
- step = 1

Example 1: Slicing a Python List

my_list = [10, 20, 30, 40, 50, 60]

```
slice1 = my_list[1:4]
print(slice1)
```

Line-by-Line Explanation:

1. my_list = [10, 20, 30, 40, 50, 60] → A list of 6 elements.
2. my_list[1:4] → Starts at index 1 (20) and goes up to index 3 (40).
    - Does not include index 4 (50)
3. Output: [20, 30, 40]

Example 2: Slicing with Step

```
my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8]
print(my_list[::2])  # Even index elements
```

Explanation:

- start: blank → start from 0
- stop: blank → go till end
- step = 2 → pick every 2nd element

Output:

[0, 2, 4, 6, 8]

Example 3: Slicing with Negative Indexes

my_list = [10, 20, 30, 40, 50]

print(my_list[-3:-1])

Explanation:

- -3 → 3rd from end (30)
- -1 → 1st from end, exclusive (50 not included)

Output:

[30, 40]

Example 4: Reversing a List

print(my_list[::-1])

Explanation:

- step = -1 → reverses the list

Output:

[50, 40, 30, 20, 10]

Example 5: Slicing NumPy Arrays

import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[1:4])

Works the same as lists:

- Output: [20 30 40]

Example 6: 2D NumPy Array Slicing

arr2d = np.array([[1, 2, 3],

[4, 5, 6],

[7, 8, 9]])

print(arr2d[0:2, 1:3])

Explanation:

- 0:2 → Rows 0 and 1
- 1:3 → Columns 1 and 2

Output:

[[2 3]

[5 6]]

Slicing a 2D NumPy Array

A **2D array** is like a matrix (rows × columns).

Example 1: Basic 2D Array

```
import numpy as np


arr2d = np.array([
        [10, 20, 30],
        [40, 50, 60],
        [70, 80, 90]
])
```

📊 **Visual Structure:**

	Col 0	**Col 1**	**Col 2**
Row 0	10	20	30
Row 1	40	50	60
Row 2	70	80	90

Example 2: Slice a block

```
print(arr2d[0:2, 1:3])
```

Line-by-Line Explanation:

- 0:2 → Rows 0 and 1 (up to, but not including, row 2)

- 1:3 → Columns 1 and 2 (up to, not including column 3)

Result:

[[20 30]

[50 60]]

Example 3: Select all rows, first column

print(arr2d[:, 0])

Explanation:

- : → All rows
- 0 → First column only

Output:

[10 40 70]

Example 4: Select last row, all columns

print(arr2d[-1, :])

Explanation:

- -1 → Last row
- : → All columns

Output:

[70 80 90]

Slicing a 3D NumPy Array

A **3D array** is like a collection of matrices (blocks of rows × columns).

Example 1: Create a 3D Array

arr3d = np.array([

[[1, 2, 3],

[4, 5, 6]],

[[7, 8, 9],

[10, 11, 12]]

])

Structure:

- Shape = (2, 2, 3)
- 2 blocks (depths), each 2 rows × 3 columns

Block 0:

[[1 2 3]

[4 5 6]]

Block 1:

[[7 8 9]

 [10 11 12]]

Example 2: Access Block 1

print(arr3d[1])

Output:

[[7 8 9]

 [10 11 12]]

Example 3: Access value 11

print(arr3d[1, 1, 1])

Explanation:

- Block 1 → second 2D array
- Row 1 → [10, 11, 12]
- Column 1 → 11

Output:

11

Example 4: Slice all blocks, last row, first column

print(arr3d[:, 1, 0])

Explanation:

- : → All blocks
- 1 → Second row
- 0 → First column

Output:

[4 10]

Example 5: Extract subarray from all blocks

print(arr3d[: , :, 1:])

Explanation:

- : → All blocks
- : → All rows
- 1: → Columns from index 1 to end

Output:

[[[2 3]
 [5 6]]

 [[8 9]
 [11 12]]]

17Q) Explain concatenation of Numpy arrays.

Ans:

Concatenation of NumPy Arrays

What is Concatenation?

Concatenation means **joining two or more NumPy arrays into a single array.**

NumPy provides different methods to concatenate arrays, such as:

- `np.concatenate()`
- `np.vstack()`
- `np.hstack()`

1. `np.concatenate()`

Definition

`np.concatenate()` is used to **join two or more arrays along a specified axis.**

Syntax

np.concatenate((array1, array2), axis=0)

- `array1, array2` → Arrays to be joined.
- `axis=0` → Joins row-wise (default).

- `axis=1` → Joins column-wise.

Example 1: Concatenate 1D Arrays

import numpy as np

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

c = np.concatenate((a, b))

print(c)

Output

[1 2 3 4 5 6]

Explanation

Both arrays are joined into one single array.

Example 2: Concatenate 2D Arrays (Row-wise)

import numpy as np

a = np.array([[1, 2],

[3, 4]])

```
b = np.array([[5, 6],
              [7, 8]])

c = np.concatenate((a, b), axis=0)

print(c)
```

Output

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Explanation

- Arrays are joined **row-wise**.
- Rows increase from **2 → 4**.
- Columns remain **2**.

Example 3: Concatenate 2D Arrays (Column-wise)

```
import numpy as np
```

```
a = np.array([[1, 2],
              [3, 4]])

b = np.array([[5, 6],
              [7, 8]])

c = np.concatenate((a, b), axis=1)

print(c)
```

Output

```
[[1 2 5 6]
 [3 4 7 8]]
```

Explanation

- Arrays are joined **column-wise**.
- Rows remain **2**.
- Columns increase from **2 → 4**.

2. `np.vstack()` (Vertical Stack)

Definition

`np.vstack()` is used to **join arrays vertically (top to bottom).**

It is the same as using:

np.concatenate((a, b), axis=0)

Syntax

np.vstack((array1, array2))

Example

import numpy as np

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

c = np.vstack((a, b))

print(c)

Output

[[1 2 3]

 [4 5 6]]

Explanation

- Arrays are stacked **one below the other.**
- The result is a **2D array.**

3. `np.hstack()` (Horizontal Stack)

Definition

`np.hstack()` is used to **join arrays horizontally (side by side)**.

It is similar to:

np.concatenate((a, b), axis=1)

(for 2D arrays)

Syntax

np.hstack((array1, array2))

Example

import numpy as np

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

c = np.hstack((a, b))

print(c)

Output

[1 2 3 4 5 6]

Explanation

The arrays are joined **side by side**, forming a longer 1D array.

`hstack()` with 2D Arrays

```
import numpy as np


a = np.array([[1, 2],
              [3, 4]])


b = np.array([[5, 6],
              [7, 8]])


c = np.hstack((a, b))


print(c)
```

Output

```
[[1 2 5 6]
 [3 4 7 8]]
```

Explanation

- Arrays are joined **side by side.**
- Number of rows stays the same.

- Number of columns increases.

Difference Between `concatenate()`, `vstack()`, and `hstack()`

Function	Meaning	Joins Along	Result
`np.concatenate()`	General function to join arrays	`axis=0` (rows) or `axis=1` (columns)	Flexible; user chooses the axis
`np.vstack()`	Vertical Stack	Rows (top to bottom)	Number of rows increases
`np.hstack()`	Horizontal Stack	Columns (side by side)	Number of columns increases

Easy Trick to Remember

- **`vstack` → V = Vertical = Top to Bottom ⬇️**
- **`hstack` → H = Horizontal = Left to Right ➡️**

Quick Revision

- **Concatenation** → Combines two or more arrays into one.

- `np.concatenate()` → Joins arrays along a specified axis (`axis=0` or `axis=1`).
- `np.vstack()` → Stacks arrays vertically (one below another).
- `np.hstack()` → Stacks arrays horizontally (side by side).

18Q) Explain Splitting Numpy arrays.

Ans:

Splitting NumPy Arrays

What is Splitting?

Splitting means **dividing one NumPy array into two or more smaller arrays.**

NumPy provides different functions to split arrays, such as:

- np.split()
- np.vsplit()
- np.hsplit()

1. np.split()

Definition

np.split() is used to **split an array into multiple equal parts.**

Syntax

np.split(array, sections, axis=0)

- **array** → Array to be split.
- **sections** → Number of equal parts.
- **axis=0** → Split row-wise (default).
- **axis=1** → Split column-wise.

Example 1: Splitting a 1D Array

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])

result = np.split(arr, 3)

print(result)
```

Output

```
[array([10, 20]),

 array([30, 40]),

 array([50, 60])]
```

Explanation

The array is divided into **3 equal parts**.

Example 2: Splitting a 2D Array (Row-wise)

```
import numpy as np

arr = np.array([[1,2],

                [3,4],

                [5,6],

                [7,8]])
```

```
result = np.split(arr, 2)
print(result)
```

Output

```
[array([[1,2],
        [3,4]]),
 array([[5,6],
        [7,8]])]
```

Explanation

- The array is split **row-wise**.
- Each part contains **2 rows**.

Example 3: Splitting a 2D Array (Column-wise)

```
import numpy as np
arr = np.array([[1,2,3,4],
                [5,6,7,8]])
result = np.split(arr, 2, axis=1)
print(result)
```

Output

```
[array([[1,2],
        [5,6]]),
```

array([[3,4],

[7,8]])]

Explanation

- The array is split **column-wise**.
- Each part contains **2 columns**.

2. np.vsplit() (Vertical Split)

Definition

np.vsplit() is used to **split a 2D array vertically (top to bottom)**.

It is the same as:

np.split(array, sections, axis=0)

Syntax

np.vsplit(array, sections)

Example

import numpy as np

arr = np.array([[1,2],

[3,4],

```
        [5,6],
        [7,8]])

result = np.vsplit(arr, 2)

print(result)
```

Output

```
[array([[1,2],
       [3,4]]),

 array([[5,6],
       [7,8]])]
```

Explanation

- The array is split **from top to bottom**.
- Each part has **2 rows**.

3. np.hsplit() (Horizontal Split)

Definition

np.hsplit() is used to **split a 2D array horizontally (left to right).**

It is the same as:

np.split(array, sections, axis=1)

Syntax

np.hsplit(array, sections)

Example

```
import numpy as np

arr = np.array([[1,2,3,4],
                [5,6,7,8]])

result = np.hsplit(arr, 2)

print(result)
```

Output

```
[array([[1,2],
        [5,6]]),
```

```
array([[3,4],
       [7,8]])]
```

Explanation

- The array is split **from left to right.**
- Each part contains **2 columns.**

Difference Between split(), vsplit(), and hsplit()

Function	Meaning	Splits Along	Result
np.split()	General function to split arrays	axis=0 (rows) or axis=1 (columns)	Flexible; user chooses the axis
np.vsplit()	Vertical Split	Rows (top to bottom)	Number of rows is divided
np.hsplit()	Horizontal Split	Columns (left to right)	Number of columns is divided

Easy Trick to Remember

- **vsplit → V = Vertical = Top to Bottom ⬇️**
- **hsplit → H = Horizontal = Left to Right ➡️**

Important Note

The array must be **divisible into equal parts**. Otherwise, NumPy raises an error.

Example:

arr = np.array([1,2,3,4,5])

np.split(arr, 2)

Output

ValueError: array split does not result in an equal division

This is because **5 elements cannot be divided equally into 2 parts.**

Quick Revision

- **Splitting** → Divides one array into smaller arrays.
- **np.split()** → General function to split arrays using axis=0 or axis=1.
- **np.vsplit()** → Splits arrays vertically (top to bottom).
- **np.hsplit()** → Splits arrays horizontally (left to right).

19Q) Explain the mathematical functions of the Numpy array.

Ans:

NumPy Mathematical Functions

What are Mathematical Functions?

Mathematical functions are built-in NumPy functions used to perform **mathematical calculations on every element of an array**.

Unlike aggregation functions (`sum()`, `mean()`), these functions **do not return a single value**. Instead, they **perform the operation on each element and return a new array**.

1. `np.abs()` / `np.absolute()` – Absolute Value

Definition

`np.abs()` or `np.absolute()` returns the **absolute (positive) value** of each element in the array.

- Positive numbers remain unchanged.
- Negative numbers become positive.

Syntax

np.abs(array)

or

np.absolute(array)

Example

import numpy as np

arr = np.array([-10, 5, -7, 8])

print(np.abs(arr))

Output

[10 5 7 8]

Explanation

- `-10 → 10`
- `5 → 5`
- `-7 → 7`
- `8 → 8`

2. Absolute Value of Complex Numbers

For **complex numbers**, `np.abs()` returns the **magnitude (distance from the origin)** of the complex number.

The magnitude is calculated as:

[
|a+bi|=\sqrt{a^2+b^2}
]

Example

import numpy as np

arr = np.array([3+4j, 5+12j])

print(np.abs(arr))

Output

[5. 13.]

Explanation

- `3 + 4j` → $\sqrt{(3^2 + 4^2)} = \mathbf{5}$
- `5 + 12j` → $\sqrt{(5^2 + 12^2)} = \mathbf{13}$

3. Exponential Functions

Exponential functions calculate the value of **e raised to the power of each element.**

`np.exp()`

Syntax

np.exp(array)

Example

import numpy as np

arr = np.array([1,2,3])

print(np.exp(arr))

Output

[2.71828183 7.3890561 20.08553692]

Explanation

- `e¹ = 2.718`
- `e² = 7.389`
- `e³ = 20.086`

4. Logarithmic Functions

NumPy provides different logarithm functions.

(a) `np.log()` – Natural Logarithm

Returns the logarithm to the base **e**.

Example

import numpy as np

arr = np.array([1, np.e, np.e**2])

print(np.log(arr))

Output

[0. 1. 2.]

(b) `np.log10()`

Returns the logarithm to **base 10**.

```
import numpy as np

arr = np.array([1,10,100,1000])

print(np.log10(arr))
```

Output

```
[0. 1. 2. 3.]
```

(c) `np.log2()`

Returns the logarithm to **base 2**.

```
import numpy as np

arr = np.array([1,2,4,8])

print(np.log2(arr))
```

Output

```
[0. 1. 2. 3.]
```

5. Trigonometric Functions

NumPy provides functions to calculate trigonometric values.

Important: NumPy expects the angle in **radians**, not degrees.

Common Functions

- `np.sin()`
- `np.cos()`
- `np.tan()`

Example

```
import numpy as np

angle = np.pi/2

print(np.sin(angle))

print(np.cos(angle))

print(np.tan(angle))
```

Output

```
1.0

6.123234e-17

1.6331239e+16
```

Explanation

- `sin(90°) = 1`
- `cos(90°) ≈ 0`
- `tan(90°)` is undefined, so Python returns a very large number due to floating-point approximation.

Degrees to Radians

If the angle is given in degrees, convert it first.

Example

```
import numpy as np
```

angle = np.deg2rad(90)

print(np.sin(angle))

Output

1.0

6. Square Root

`np.sqrt()`

Returns the square root of each element.

Example

import numpy as np

arr = np.array([4,9,16,25])

print(np.sqrt(arr))

Output

[2. 3. 4. 5.]

7. Power Function

`np.power()`

Raises each element to a specified power.

Example

import numpy as np

arr = np.array([2,3,4])

```
print(np.power(arr,2))
```

Output

```
[ 4  9 16]
```

8. Rounding Functions

`np.round()`

Rounds values to the nearest integer (or specified decimal places).

```
import numpy as np

arr = np.array([2.4,2.8,3.6])
print(np.round(arr))
```

Output

```
[2. 3. 4.]
```

9. Ceiling Function

`np.ceil()`

Rounds every value **up** to the nearest integer.

```
import numpy as np
arr = np.array([2.1,3.4,4.8])
print(np.ceil(arr))
```

Output

[3. 4. 5.]

10. Floor Function

`np.floor()`

Rounds every value **down** to the nearest integer.

import numpy as np

arr = np.array([2.9,3.7,4.2])

print(np.floor(arr))

Output

[2. 3. 4.]

Summary Table

Function	Purpose
`np.abs()` / `np.absolute()`	Returns the absolute (positive) value of each element
`np.abs()` (complex numbers)	Returns the magnitude of complex numbers
`np.exp()`	Calculates e raised to each element
`np.log()`	Natural logarithm (base e)

`np.log10()`	Logarithm to base 10
`np.log2()`	Logarithm to base 2
`np.sin()`	Calculates sine of each angle (in radians)
`np.cos()`	Calculates cosine of each angle (in radians)
`np.tan()`	Calculates tangent of each angle (in radians)
`np.sqrt()`	Returns the square root of each element
`np.power()`	Raises each element to a specified power
`np.round()`	Rounds values to the nearest integer/decimal
`np.ceil()`	Rounds values upward
`np.floor()`	Rounds values downward

20Q) Explain Sorting in Numpy.

Ans:

Sorting in NumPy Array

What is Sorting?

Sorting is the process of **arranging the elements of an array in a particular order.**

The order can be:

- **Ascending order** (smallest to largest)
- **Descending order** (largest to smallest)

NumPy provides the `np.sort()` function to sort arrays.

`np.sort()`

Definition

`np.sort()` is used to **sort the elements of a NumPy array** in **ascending order** by default.

Syntax

np.sort(array)

Example 1: Sorting a 1D Array

import numpy as np

arr = np.array([50, 20, 40, 10, 30])

sorted_arr = np.sort(arr)

print(sorted_arr)

Output

[10 20 30 40 50]

Explanation

The elements are arranged from **smallest to largest**.

Example 2: Sorting a String Array

```
import numpy as np

arr = np.array(["banana", "apple", "mango"])

print(np.sort(arr))
```

Output

['apple' 'banana' 'mango']

Explanation

Strings are sorted in **alphabetical order**.

Example 3: Sorting a 2D Array

```
import numpy as np

arr = np.array([[3, 1, 2],
```

[6, 4, 5]])

print(np.sort(arr))

Output

[[1 2 3]

[4 5 6]]

Explanation

By default, `np.sort()` sorts **each row separately**.

- First row: `[3, 1, 2]` → `[1, 2, 3]`
- Second row: `[6, 4, 5]` → `[4, 5, 6]`

Sorting Along an Axis

For a 2D array, you can choose whether to sort **rows** or **columns.**

`axis=1` (Default) → Sort Each Row

import numpy as np

arr = np.array([[3, 1, 2],

[6, 4, 5]])

```
print(np.sort(arr, axis=1))
```

Output

```
[[1 2 3]
 [4 5 6]]
```

Explanation: Each row is sorted separately.

`axis=0` → Sort Each Column

```
import numpy as np

arr = np.array([[3, 1, 2],
                [6, 4, 5]])

print(np.sort(arr, axis=0))
```

Output

```
[[3 1 2]
 [6 4 5]]
```

Explanation

Each column is sorted separately.

Let's see each column:

Before	After
3, 6	3, 6
1, 4	1, 4
2, 5	2, 5

Since the columns were already in ascending order, the array remains unchanged.

Better Example

```
arr = np.array([[6, 4, 5],
                [3, 1, 2]])


print(np.sort(arr, axis=0))
```

Output

```
[[3 1 2]
 [6 4 5]]
```

Now each **column** is sorted from **top to bottom**.

Descending Order

`np.sort()` sorts in **ascending order only**.

To get the descending **order**, reverse the sorted array.

import numpy as np

arr = np.array([50, 20, 40, 10, 30])

print(np.sort(arr)[::-1])

Output

[50 40 30 20 10]

Important Points

- `np.sort()` returns a **new sorted array**.
- The **original array is not changed**.
- By default, sorting is done in **ascending order**.

Quick Revision

Function	Purpose

`np.sort(arr)`	Sorts the array in ascending order
`axis=0`	Sorts each column
`axis=1`	Sorts each row (default for 2D arrays)
`[::-1]`	Reverses the sorted array to get descending order

21Q) Explain Boolean array and Masking in Numpy.

Ans:

Boolean indexing and masking are among the most confusing NumPy topics for beginners. The easiest way to understand them is to think of them as **a filter**.

What is a Boolean Array?

A **Boolean array** is an array that contains only **True** and **False** values.

These True/False values are usually created by comparing array elements with a condition.

For example,

import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr > 25)

Output

[False False True True True]

How did this happen?

NumPy checks **each element one by one.**

Element	Condition (>25)	Result
10	10 > 25	False
20	20 > 25	False
30	30 > 25	True
40	40 > 25	True
50	50 > 25	True

So NumPy creates a **Boolean array**.

[False False True True True]

What is Masking?

A **mask** is simply a Boolean array.

It acts like a **filter.**

- **True** → Keep this element ✅
- **False** → Ignore this element ❌

That's why it's called a **mask.**

Imagine you're selecting students who scored more than 25 marks.

Marks	Selected?
10	❌
20	❌
30	✅
40	✅
50	✅

The mask is

[False False True True True]

Example 1

import numpy as np

```
arr = np.array([10,20,30,40,50])

mask = arr > 25


print(mask)
```

Output

```
[False False True True True]
```

Now use the mask.

```
print(arr[mask])
```

Output

```
[30 40 50]
```

What happened?

Original array

```
[10 20 30 40 50]
```

Mask

```
[False False True True True]
```

NumPy checks them together.

Array	Mask	Result
10	False	Remove
20	False	Remove
30	True	Keep
40	True	Keep
50	True	Keep

Final output

[30 40 50]

Example 2

Find even numbers.

arr = np.array([1,2,3,4,5,6,7,8])

mask = arr % 2 == 0

print(mask)

Output

[False True False True False True False True]

Now apply the mask.

print(arr[mask])

Output

[2 4 6 8]

Step-by-step

Number	Even?	Mask
1	No	False

2	Yes	True
3	No	False
4	Yes	True
5	No	False
6	Yes	True
7	No	False
8	Yes	True

Only True values remain.

Example 3

Select numbers less than 50.

arr = np.array([25,45,60,70,15])

print(arr[arr<50])

Output

[25 45 15]

Why don't we write loops?

Without NumPy

```
for i in arr:
    if i>25:
        print(i)
```

With NumPy

```
arr[arr>25]
```

Much shorter and much faster.

Common Comparison Operators

Operator	Meaning
>	Greater than

`<`	Less than
`>=`	Greater than or equal to
`<=`	Less than or equal to
`==`	Equal to
`!=`	Not equal to

Multiple Conditions

AND (&)

Select numbers greater than 20 **and** less than 50.

arr = np.array([10,20,30,40,50,60])

print(arr[(arr>20) & (arr<50)])

Output

[30 40]

OR (|)

Select numbers less than 20 **or** greater than 50.

```
print(arr[(arr<20) | (arr>50)])
```

Output

```
[10 60]
```

Real-life Example

Imagine a class.

Student	Marks
A	35
B	82
C	27
D	91
E	45

You want only students with marks above 50.

Boolean array

[False True False True False]

Mask

❌✅❌✅❌

Selected marks

[82 91]

One-line Definitions (Exam)

Boolean Array

> A **Boolean array** is an array containing only **True** and **False** values, usually created by applying a condition to a NumPy array.

Masking

> **Masking** is the process of using a Boolean array to filter and select specific elements from a NumPy array.

Quick Revision

Original Array

[10 20 30 40 50]

Condition

arr $>$ 25

↓

Boolean Array

[False False True True True]

↓

Mask Applied

10 ❌

20 ❌

30 ✅

40 ✅

50 ✅

↓

Result

[30 40 50]

Memory Trick

Think of a **Boolean mask as a security guard**:

- **True** → "You are allowed in." ✅
- **False** → "You are not allowed in." ❌

The guard checks every element in the array and lets only the **True** ones pass. This is exactly how Boolean masking works in NumPy.

22Q) Explain Broadcasting in Numpy.

Ans:

Broadcasting in NumPy

What is Broadcasting?

Broadcasting is a feature in NumPy that allows us to **perform mathematical operations on arrays of different shapes without writing loops.**

In simple words,

> **Broadcasting means NumPy automatically adjusts the size of the smaller array so that operations can be performed with the larger array.**

It makes the code **shorter, faster, and easier to write.**

Why is Broadcasting Needed?

Suppose you have an array:

import numpy as np

arr = np.array([10, 20, 30, 40])

You want to add **5** to every element.

Without broadcasting, you would write:

[10+5, 20+5, 30+5, 40+5]

With broadcasting, simply write:

arr + 5

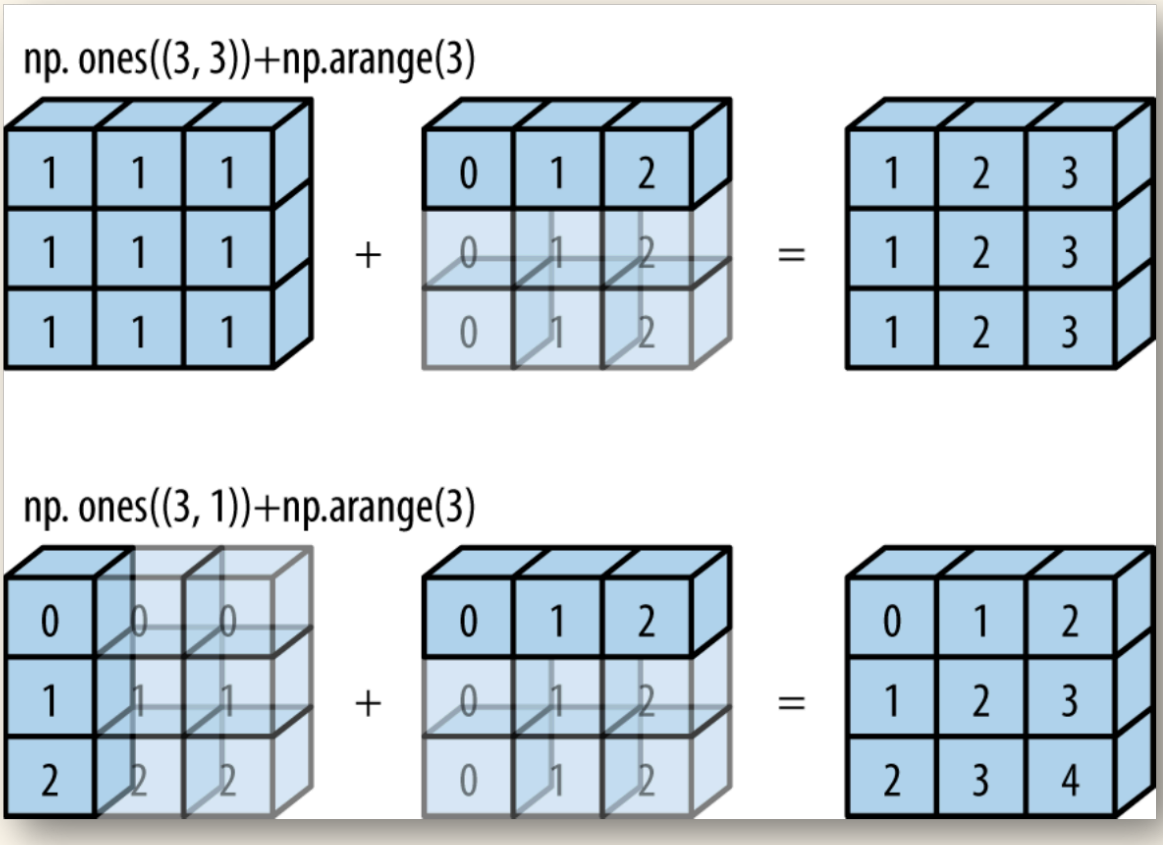
Output

[15 25 35 45]

NumPy automatically treats 5 as:

[5 5 5 5]

This automatic expansion is called **broadcasting**.



Example 1: Scalar Broadcasting

import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr + 10)

Output

[11 12 13 14]

What happens internally?

Original Array

[1 2 3 4]

+

10

↓

NumPy treats it as

[10 10 10 10]

↓

Result

[11 12 13 14]

Example 2: Two Arrays

```
import numpy as np

a = np.array([10,20,30])

b = np.array([1,2,3])

print(a+b)
```

Output

```
[11 22 33]
```

Each element is added to the corresponding element.

$10 + 1 = 11$

$20 + 2 = 22$

$30 + 3 = 33$

Example 3: 2D Array and 1D Array

```
import numpy as np


a = np.array([[1,2,3],
```

```
        [4,5,6]])

b = np.array([10,20,30])

print(a+b)
```

Output

```
[[11 22 33]
 [14 25 36]]
```

How?

Original arrays

a

1 2 3

4 5 6

b

10 20 30

NumPy automatically copies b for every row.

10 20 30

10 20 30

Now addition becomes

1+10 2+20 3+30

4+10 5+20 6+30

Result

11 22 33

14 25 36

Rules of Broadcasting

NumPy follows **three simple rules.**

Rule 1: Dimensions must be the same OR one array must have fewer dimensions.

Example

a = np.array([[1,2,3],

[4,5,6]]) #(2,3)

b = np.array([10,20,30]) #(3,)

Since NumPy can treat `(3,)` as `(1,3)`, broadcasting is possible.

Rule 2: Compare Shapes from Right to Left

NumPy always compares dimensions starting from the **last (rightmost) dimension**.

Example

(2,3)

(3,)

Treat `(3,)` as

(1,3)

Comparison

2 3

1 3

- 3 = 3 ✅
- 2 and 1 → allowed because one dimension is 1 ✅

So broadcasting works.

Rule 3: Two Dimensions are Compatible If

For every dimension,

- They are **equal**, OR
- One of them is **1**

Otherwise, broadcasting is **not possible**.

Example (Valid)

Shape A = (2,3)

Shape B = (1,3)

Comparison

2 3

1 3

- 3 = 3 ✅
- 2 and 1 ✅

Broadcasting works.

Example (Invalid)

Shape A = (2,3)

Shape B = (2,2)

Comparison

2 3

2 2

- 3 ≠ 2 ❌
- Neither is 1 ❌

Broadcasting fails.

Example

import numpy as np

```
a = np.array([[1,2,3],
              [4,5,6]])

b = np.array([[10,20],
              [30,40]])

print(a+b)
```

Output

ValueError:

operands could not be broadcast together

Another Example

```
a = np.array([[1],
              [2],
              [3]])      #(3,1)

b = np.array([10,20,30]) #(3,)
```

Shapes

(3,1)

(1,3)

NumPy broadcasts them as

1 10 20 30

2 10 20 30

3 10 20 30

Result

[[11 21 31]

[12 22 32]

[13 23 33]]

Advantages of Broadcasting

- No need to write loops.
- Makes the code shorter and cleaner.
- Faster than using loops.
- Saves memory because NumPy does not actually copy the data.

Quick Revision

Rule	Meaning
Rule 1	Arrays should have the same number of dimensions, or the smaller array should have fewer dimensions.
Rule 2	Compare array shapes from **right to left**.
Rule 3	Two dimensions are compatible if they are **equal** or **one of them is 1**. Otherwise, broadcasting is not possible.

Memory Trick

Imagine **a teacher giving notebooks to students**.

- If there is **one notebook**, the teacher can **give a copy to every student**. ✔️
- If there are **exactly as many notebooks as students**, each student gets one. ✔️
- If there are **3 notebooks for 5 students**, the teacher cannot distribute them equally. ❌

Similarly, in broadcasting:

- **Dimension = 1** → NumPy can automatically repeat it.
- **Dimensions are equal** → NumPy matches them directly.

- **Dimensions are different and neither is 1** → Broadcasting fails.

23Q) Write datatypes in Numpy.

Ans:

Character	Description	Example
'b'	Byte	np.dtype('b')
'i'	Signed integer	np.dtype('i4') == np.int32
'u'	Unsigned integer	np.dtype('u1') == np.uint8
'f'	Floating point	np.dtype('f8') == np.int64
'c'	Complex floating point	np.dtype('c16') == np.complex128
'S','a'	string	np.dtype('S5')
'U'	Unicode string	np.dtype('U') == np.str_
'V'	Raw data (void)	np.dtype('V') == np.void

24Q) What is Linear Algebra in Numpy.

Ans:

What is Linear Algebra?

Linear algebra deals with vectors, matrices, and systems of linear equations. NumPy provides a special module for this:

import numpy.linalg

Or use:

from numpy import linalg as LA

Common NumPy Linear Algebra Functions

Function	Description
np.dot(a, b)	Matrix or dot product
np.matmul(a, b)	Matrix multiplication
np.linalg.inv(a)	Inverse of matrix
np.linalg.det(a)	Determinant of matrix
np.linalg.eig(a)	Eigenvalues and eigenvectors
np.linalg.solve(A, b)	Solves Ax = b
np.linalg.norm(a)	Vector/matrix norm (magnitude)
np.transpose(a)	Transpose of matrix

import numpy as np

Step 1: Define a 2x2 matrix A and vector b

A = np.array([[2, 1],

```
                [1, 3]])

b = np.array([8, 13])

# Step 2: Solve Ax = b
x = np.linalg.solve(A, b)

# Step 3: Compute the inverse of A
A_inv = np.linalg.inv(A)

# Step 4: Verify the solution using dot product
check = np.dot(A, x)

# Step 5: Calculate determinant of A
det = np.linalg.det(A)

# Step 6: Find eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)

# Step 7: Compute the norm (length) of vector b
```

```
b_norm = np.linalg.norm(b)


# Step 8: Transpose of matrix A
A_T = A.T


# Display results
print("Matrix A:\n", A)
print("Vector b:", b)
print("Solution x:", x)
print("Inverse of A:\n", A_inv)
print("Verification Ax:", check)
print("Determinant of A:", det)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
print("Norm of b:", b_norm)
print("Transpose of A:\n", A_T)
```

Line-by-Line Explanation

Step 1: Define Matrix and Vector

```
A = np.array([[2, 1],
```

[1, 3]])

b = np.array([8, 13])

- A is a 2×2 matrix
- b is a vector in the system Ax = b

Step 2: Solve the System Ax = b

x = np.linalg.solve(A, b)

- Uses **matrix algebra** to solve for x
- Internally, it avoids calculating the inverse for numerical stability

Output:

x = [3. 2.]

2*3 + 1*2 = 8

1*3 + 3*2 = 13

Step 3: Inverse of Matrix

A_inv = np.linalg.inv(A)

- Computes $\mathbf{A^{-1}}$, such that A @ A^{-1} = I (identity)

Output:

[[0.6 -0.2]

 [-0.2 0.4]]

Step 4: Verify Ax = b

check = np.dot(A, x)

- Calculates matrix multiplication: A * x → should equal b

Output:

[8. 13.]

Step 5: Determinant of A

det = np.linalg.det(A)

- **det(A)** tells us if matrix is invertible (det ≠ 0)

Output:

5.0

Step 6: Eigenvalues and Eigenvectors

eigenvalues, eigenvectors = np.linalg.eig(A)

- Solves for:
    - λ (eigenvalues)
    - v (eigenvectors)

Output (example):

Eigenvalues: [1.3819, 3.6180]

Eigenvectors:

[[-0.8507 -0.5257]

[0.5257 -0.8507]]

Step 7: Norm (Length) of Vector

b_norm = np.linalg.norm(b)

- Computes the magnitude of b using:
  $\sqrt{(8^2 + 13^2)}$

Output:

15.2643

Step 8: Transpose of Matrix

A_T = A.T

- Transposes the matrix: rows ↔ columns

Output:

[[2 1]

[1 3]]

Line-by-Line Explanation

Step 1: Define Matrix and Vector

A = np.array([[2, 1],

[1, 3]])

b = np.array([8, 13])

- A is a 2×2 matrix
- b is a vector in the system $Ax = b$

Step 2: Solve the System Ax = b

x = np.linalg.solve(A, b)

- Uses **matrix algebra** to solve for x
- Internally, it avoids calculating the inverse for numerical stability

Output:

x = [3. 2.]

Which means:

markdown

2*3 + 1*2 = 8

1*3 + 3*2 = 13

Step 3: Inverse of Matrix

A_inv = np.linalg.inv(A)

- Computes $\mathbf{A^{-1}}$, such that A @ A^{-1} = I (identity)

Output:

[[0.6 -0.2]

 [-0.2 0.4]]

Step 4: Verify Ax = b

check = np.dot(A, x)

- Calculates matrix multiplication: A * x → should equal b

Output:

[8. 13.]

Step 5: Determinant of A

det = np.linalg.det(A)

- **det(A)** tells us if matrix is invertible (det ≠ 0)

Output:

5.0

Step 6: Eigenvalues and Eigenvectors

eigenvalues, eigenvectors = np.linalg.eig(A)

- Solves for:
  - λ (eigenvalues)
  - v (eigenvectors)

Output (example):

Eigenvalues: [1.3819, 3.6180]

Eigenvectors:

[[-0.8507 -0.5257]

 [0.5257 -0.8507]]

Step 7: Norm (Length) of Vector

b_norm = np.linalg.norm(b)

- Computes the **magnitude** of b using:
  $\sqrt{(8^2 + 13^2)}$

Output:

15.2643

Step 8: Transpose of Matrix

A_T = A.T

- Transposes the matrix: rows ↔ columns

Output:

[[2 1]

[1 3]]

25Q) What is np.reshape() and np.flatten().

Ans:

Reshape and Flatten in NumPy

Both `reshape()` and `flatten()` are used to **change the shape of a NumPy array**, but they work differently.

- `reshape()` changes the shape of an array **without changing its data.**
- `flatten()` converts a multi-dimensional array into a **1D array.**

1. `reshape()`

Definition

`reshape()` is used to **change the shape (dimensions) of an array without changing its elements.**

The total number of elements **must remain the same** before and after reshaping.

Syntax

array.reshape(rows, columns)

Example 1: Convert 1D Array to 2D Array

import numpy as np

arr = np.array([1,2,3,4,5,6])

new_arr = arr.reshape(2,3)

print(new_arr)

Output

[[1 2 3]

[4 5 6]]

Explanation

Original array

[1 2 3 4 5 6]

After reshaping to `(2,3)`

1 2 3

4 5 6

- **2 rows**
- **3 columns**

Example 2: Convert 1D Array to 3D Array

import numpy as np

arr = np.array([1,2,3,4,5,6,7,8])

```
new_arr = arr.reshape(2,2,2)

print(new_arr)
```

Output

```
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Explanation

Shape becomes

(2,2,2)

2 layers

2 rows

2 columns

Important Rule

The **total number of elements must remain the same.**

Example

arr = np.array([1,2,3,4,5,6])

arr.reshape(3,2)

Works because

$3 \times 2 = 6$ elements

Invalid Example

arr.reshape(4,2)

Error

ValueError:

cannot reshape array of size 6 into shape (4,2)

Because

$4 \times 2 = 8$

But array has only 6 elements.

Using `-1` in `reshape()`

NumPy can automatically calculate one dimension.

Example

arr = np.array([1,2,3,4,5,6])

print(arr.reshape(2,-1))

Output

[[1 2 3]

 [4 5 6]]

Here,

- Number of rows = 2
- NumPy automatically calculates the number of columns = 3.

2. `flatten()`

Definition

`flatten()` is used to **convert a multi-dimensional array into a one-dimensional (1D) array.**

Syntax

array.flatten()

Example

```
import numpy as np


arr = np.array([[1,2,3],
                [4,5,6]])


print(arr.flatten())
```

Output

[1 2 3 4 5 6]

Explanation

Original array

1 2 3

4 5 6

After flattening

[1 2 3 4 5 6]

Everything becomes a single row.

Example with 3D Array

```
arr = np.array([
    [[1,2],[3,4]],
    [[5,6],[7,8]]
])

print(arr.flatten())
```

Output

[1 2 3 4 5 6 7 8]

Difference Between `reshape()` and `flatten()`

Feature	`reshape()`	`flatten()`

Purpose	Changes the shape of an array	Converts an array into a 1D array
Dimensions	Can convert to any valid shape	Always returns a 1D array
Number of elements	Remains the same	Remains the same
Output	Array with a new shape	One-dimensional array

Visual Representation

Original Array

1 2 3

4 5 6

Using `reshape(3,2)`

1 2

3 4

5 6

Only the **shape changes.**

Using `flatten()`

[1 2 3 4 5 6]

Everything becomes **one single row.**

Easy Trick to Remember

`reshape()`

Think of **rearranging books on a shelf.**

The books remain the same, but you arrange them differently.

Before

1 2 3 4 5 6

↓

After

1 2 3

4 5 6

`flatten()`

Think of **flattening a cardboard box.**

A 3D box becomes a flat sheet.

Similarly,

1 2 3

4 5 6

↓

1 2 3 4 5 6

Quick Revision

Function	Purpose
`reshape()`	Changes the shape of an array without changing its elements
`flatten()`	Converts a multi-dimensional array into a one-dimensional array

`reshape(-1)`	NumPy automatically calculates the missing dimension

26Q)Explain fancy indexing in numpy

Ans:

Fancy Indexing in NumPy

What is Fancy Indexing?

Fancy indexing is a method of **accessing one or more elements from a NumPy array using a list or an array of indices.**

Unlike normal indexing, where we access **one element at a time**, fancy indexing allows us to **access multiple elements at once.**

Why Use Fancy Indexing?

Suppose you have an array:

import numpy as np

arr = np.array([10, 20, 30, 40, 50])

If you want only **10, 30, and 50**, using normal indexing you would write:

```
print(arr[0])

print(arr[2])

print(arr[4])
```

With **fancy indexing**, you can access all of them in a single statement:

```
print(arr[[0, 2, 4]])
```

Output

```
[10 30 50]
```

Fancy Indexing in a 1D Array

Example

```
import numpy as np


arr = np.array([10, 20, 30, 40, 50])


print(arr[[1, 3, 4]])
```

Output

```
[20 40 50]
```

Explanation

NumPy selects the elements at indices:

- Index 1 → 20
- Index 3 → 40
- Index 4 → 50

Visual Representation

Index : 0 1 2 3 4

Array : 10 20 30 40 50

↑ ↑ ↑

Indices = [1, 3, 4]

Result = [20, 40, 50]

Fancy Indexing in a 2D Array

For a **2D array**, we can select **specific rows**.

Example

```
import numpy as np
```

```
arr = np.array([[10,20],
                [30,40],
                [50,60]])

print(arr[[0,2]])
```

Output

```
[[10 20]
 [50 60]]
```

Explanation

The selected rows are:

- Row 0 → `[10, 20]`
- Row 2 → `[50, 60]`

Selecting Individual Elements from a 2D Array

Use two index lists:

array[[row_indices], [column_indices]]

Example

```
import numpy as np


arr = np.array([[10,20,30],
                [40,50,60],
                [70,80,90]])


print(arr[[0,2], [1,2]])
```

Output

```
[20 90]
```

Explanation

NumPy matches the indices **pair by pair**:

Row Index	Column Index	Selected Element
0	1	20
2	2	90

Result:

```
[20 90]
```

Fancy Indexing Using a NumPy Array

Instead of a Python list, you can also use a NumPy array of indices.

Example

```
import numpy as np


arr = np.array([10,20,30,40,50])


index = np.array([0,2,4])


print(arr[index])
```

Output

```
[10 30 50]
```

Difference Between Normal Indexing and Fancy Indexing

Normal Indexing	Fancy Indexing
Accesses one element at a time	Accesses multiple elements at once

Uses a single index	Uses a list or array of indices
Example: `arr[2]`	Example: `arr[[0,2,4]]`

Advantages of Fancy Indexing

- Access multiple elements in a single statement.
- Reduces the need for loops.
- Makes the code shorter and easier to read.
- Useful for selecting specific rows or elements from large datasets.

Quick Revision

Operation	Example
Select multiple elements	`arr[[0,2,4]]`
Select multiple rows	`arr[[0,2]]`
Select specific elements in a 2D array	`arr[[0,2],[1,2]]`

Easy Trick to Remember

Imagine an array as a row of lockers:

Index : 0 1 2 3 4

Locker: A B C D E

If you give NumPy the list:

[0, 2, 4]

It opens **locker 0, locker 2, and locker 4** and returns:

A C E

This is exactly what **fancy indexing** does—it lets you **pick multiple positions at the same time.**

Viva Questions

Q1. What is fancy indexing in NumPy?
Answer: Fancy indexing is a method of accessing multiple elements of a NumPy array using a list or array of indices.

Q2. What is the main advantage of fancy indexing?
Answer: It allows multiple elements or rows to be selected in a single statement without using loops.

Q3. Can fancy indexing be used with 2D arrays?
Answer: Yes. It can be used to select specific rows or specific elements from a 2D array.

Q4. Which data types can be used for fancy indexing?
Answer: A Python list or a NumPy array of indices can be used.

Q5. What is the difference between normal indexing and fancy indexing?
Answer: Normal indexing accesses one element at a time, while fancy indexing accesses multiple elements at once using a list or array of indices.

27Q)Explain different ways to sort a NumPy array.

Ans:

Different Ways to Sort a NumPy Array

What is Sorting?

Sorting is the process of **arranging the elements of an array in a specific order**, such as:

- **Ascending order** (smallest to largest)
- **Descending order** (largest to smallest)

NumPy provides several ways to sort arrays.

1. Sorting Using `np.sort()`

Definition

`np.sort()` returns a **new sorted array** in ascending order.

Syntax

np.sort(array)

Example

import numpy as np

arr = np.array([40, 10, 30, 20])

print(np.sort(arr))

Output

[10 20 30 40]

Explanation

The elements are arranged from **smallest to largest.**

2. Sorting in Descending Order

NumPy sorts in ascending order by default.

To sort in **descending order**, reverse the sorted array using `[::-1]`.

Example

```
import numpy as np


arr = np.array([40,10,30,20])


print(np.sort(arr)[::-1])
```

Output

```
[40 30 20 10]
```

Explanation

- First, `np.sort()` sorts the array in ascending order.
- Then, `[::-1]` reverses the array to get descending order.

3. Sorting Along Rows (`axis=1`)

For a **2D array**, `axis=1` sorts each **row** separately.

Example

```
import numpy as np
```

```
arr = np.array([[3,1,2],
                [6,4,5]])

print(np.sort(arr, axis=1))
```

Output

```
[[1 2 3]
 [4 5 6]]
```

Explanation

- First row: `[3,1,2]` → `[1,2,3]`
- Second row: `[6,4,5]` → `[4,5,6]`

4. Sorting Along Columns (`axis=0`)

`axis=0` sorts each **column** separately.

Example

```
import numpy as np

arr = np.array([[6,4,5],
```

```
            [3,1,2]])

print(np.sort(arr, axis=0))
```

Output

```
[[3 1 2]
 [6 4 5]]
```

Explanation

Each column is sorted from **top to bottom.**

Before	After
6, 3	3, 6
4, 1	1, 4
5, 2	2, 5

5. Sorting Using `argsort()`

Definition

`np.argsort()` **does not return the sorted values.** Instead, it returns the **indices (positions)** that would sort the array.

Syntax

np.argsort(array)

Example

import numpy as np

arr = np.array([40,10,30,20])

print(np.argsort(arr))

Output

[1 3 2 0]

Explanation

Original array:

Index	Value
0	40
1	10
2	30
3	20

Sorted values are:

10 20 30 40

Their original indices are:

1 3 2 0

So `argsort()` returns:

[1 3 2 0]

6. Sorting Strings

`np.sort()` can also sort strings in **alphabetical order**.

Example

```
import numpy as np

arr = np.array(["Mango", "Apple", "Banana"])

print(np.sort(arr))
```

Output

```
['Apple' 'Banana' 'Mango']
```

7. Sorting Boolean Arrays

Boolean values are sorted as:

False < True

Example

```
import numpy as np

arr = np.array([True, False, True, False])

print(np.sort(arr))
```

Output

[False False True True]

Difference Between `sort()` and `argsort()`

`np.sort()`	`np.argsort()`
Returns the sorted values	Returns the indices of the sorted values
Used to arrange elements	Used to know the original positions of sorted elements

Summary Table

Method	Purpose
`np.sort(arr)`	Sorts the array in ascending order
`np.sort(arr)[::-1]`	Sorts the array in descending order

`np.sort(arr, axis=1)`	Sorts each row of a 2D array
`np.sort(arr, axis=0)`	Sorts each column of a 2D array
`np.argsort(arr)`	Returns the indices that would sort the array

Quick Revision

- **`np.sort()`** → Sorts values in ascending order.
- **`[::-1]`** → Reverses the sorted array to get descending order.
- **`axis=1`** → Sorts each row.
- **`axis=0`** → Sorts each column.
- **`np.argsort()`** → Returns the indices of the sorted elements.

Viva Questions

Q1. What is sorting in NumPy?
Answer: Sorting is the process of arranging the elements of an array in ascending or descending order.

Q2. Which function is used to sort a NumPy array?
Answer: `np.sort()`.

Q3. How do you sort an array in descending order?
Answer: Use `np.sort(arr)[::-1]`.

Q4. What is the difference between `np.sort()` and `np.argsort()`?
Answer: `np.sort()` returns the sorted values, while `np.argsort()` returns the indices of the sorted values.

Q5. What does `axis=0` mean in sorting?
Answer: It sorts each column separately.

Q6. What does `axis=1` mean in sorting?
Answer: It sorts each row separately.

28Q)Develop a Python program without using the sum function to find the sum of all the elements in a NumPy array.

Ans:

Python Program to Find the Sum of All Elements in a NumPy Array (Without Using `sum()`)

```
import numpy as np



# Create a NumPy array

arr = np.array([10, 20, 30, 40, 50])
```

```
# Initialize sum variable
total = 0

# Find sum using a loop
for i in arr:
    total = total + i

# Display the result
print("Array:", arr)
print("Sum of all elements:", total)
```

Output

```
Array: [10 20 30 40 50]
Sum of all elements: 150
```

Explanation

- `import numpy as np` → Imports the NumPy library.
- `np.array([...])` → Creates a NumPy array.
- `total = 0` → Initializes the sum variable.
- `for i in arr:` → Loops through each element of the array.
- `total = total + i` → Adds each element to `total`.

- `print()` → Displays the array and the final sum.

Algorithm

1. Import the NumPy library.
2. Create a NumPy array.
3. Initialize a variable `total` to 0.
4. Traverse each element of the array using a `for` loop.
5. Add each element to `total`.
6. Display the sum.

Time Complexity: O(n), where **n** is the number of elements in the array.

29Q)

A)How to create an empty and a full NumPy array.

B)With an example, explain how to make a NumPy array read-only.

Ans:

(A) How to Create an Empty and a Full NumPy Array

1. Empty Array

Definition

An **empty array** is an array that is **created with a specified shape, but its elements are not initialized.** The values stored in the array are random (garbage) values already present in memory.

NumPy provides the `np.empty()` function to create an empty array.

Syntax

np.empty(shape, dtype)

- **shape** → Size of the array.
- **dtype** → Data type of the array (optional).

Example

import numpy as np

arr = np.empty((2, 3))

print(arr)

Sample Output

[[6.9e-310 6.9e-310 6.9e-310]

[6.9e-310 6.9e-310 6.9e-310]]

Note: The values may be different on every system because they are **uninitialized memory values**.

2. Full Array

Definition

A **full array** is an array in which **every element is filled with the same specified value**.

NumPy provides the `np.full()` function to create a full array.

Syntax

np.full(shape, fill_value)

- **shape** → Size of the array.
- **fill_value** → Value to fill in every element.

Example

import numpy as np

arr = np.full((2, 3), 5)

print(arr)

Output

[[5 5 5]

[5 5 5]]

Explanation

- Creates a **2 × 3** array.
- Every element is filled with **5**.

Difference Between `np.empty()` and `np.full()`

Function	Description
`np.empty()`	Creates an array with **uninitialized (random)** values.
`np.full()`	Creates an array where **all elements have the same specified value**.

(B) How to Make a NumPy Array Read-Only

Definition

A **read-only array** is an array whose elements **cannot be modified** after it is created.

NumPy allows us to make an array read-only using the **`flags.writeable`** attribute.

Syntax

array.flags.writeable = False

Example

```
import numpy as np

# Create an array
arr = np.array([10, 20, 30, 40])

# Make the array read-only
arr.flags.writeable = False

# Display the array
print(arr)

# Try to modify an element
arr[0] = 100
```

Output

[10 20 30 40]

ValueError: assignment destination is read-only

Explanation

- `arr.flags.writeable = False` makes the array **read-only**.
- After this, any attempt to change an element will raise a **ValueError**.
- This helps protect the array from accidental modifications.

Viva Questions

Q1. Which function is used to create an empty array in NumPy?
Answer: `np.empty()`.

Q2. Which function is used to create a full array?
Answer: `np.full()`.

Q3. What is an empty array?
Answer: An empty array is an array whose elements are **not initialized** and contain random values from memory.

Q4. What is a full array?
Answer: A full array is an array in which **all elements are filled with the same specified value**.

Q5. How do you make a NumPy array read-only?
Answer: By setting `array.flags.writeable = False`.

Q6. What happens if you try to modify a read-only array?
Answer: NumPy raises a **`ValueError`** because the array cannot be modified.

30Q)

A)How to build an array of all combinations of two NumPy arrays.

B)Describe the process of changing data type of given NumPy array.

Ans:

(A) How to Build an Array of All Combinations of Two NumPy Arrays

Definition

Building an array of all combinations means **combining every element of the first array with every element of the second array**.

This is useful when we want to generate **all possible pairs** from two arrays.

One simple way is to use **nested loops**.

Example

```
import numpy as np


# Create two arrays
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5])


# Store all combinations
result = []


for i in arr1:
    for j in arr2:
        result.append((i, j))


print(result)
```

Output

```
[(1, 4), (1, 5),
 (2, 4), (2, 5),
```

(3, 4), (3, 5)]

Explanation

- `arr1 = [1, 2, 3]`
- `arr2 = [4, 5]`

The combinations are:

- (1, 4)
- (1, 5)
- (2, 4)
- (2, 5)
- (3, 4)
- (3, 5)

Every element of the first array is paired with every element of the second array.

Algorithm

1. Create two NumPy arrays.
2. Create an empty list.
3. Use nested loops to pair each element of the first array with each element of the second array.
4. Store each pair in the list.
5. Display the result.

(B) Describe the Process of Changing the Data Type of a NumPy Array

Definition

Changing the data type means **converting the elements of a NumPy array from one data type to another**, such as:

- Integer (`int`) → Float (`float`)
- Float (`float`) → Integer (`int`)
- Integer (`int`) → String (`str`)

NumPy provides the **`astype()`** function to change the data type of an array.

Syntax

array.astype(new_data_type)

Example 1: Convert Integer to Float

import numpy as np

arr = np.array([1, 2, 3, 4])

new_arr = arr.astype(float)

print(new_arr)

print(new_arr.dtype)

Output

[1. 2. 3. 4.]

float64

Explanation

- The original array contains integers.
- `astype(float)` converts all elements into floating-point numbers.

Example 2: Convert Float to Integer

import numpy as np

arr = np.array([1.5, 2.8, 3.9])

new_arr = arr.astype(int)

```
print(new_arr)
```

Output

```
[1 2 3]
```

Explanation

The decimal part is removed when converting from float to integer.

Important Points

- `astype()` returns a **new array** with the changed data type.
- The **original array remains unchanged**.
- You can convert to data types like `int`, `float`, `str`, `bool`, etc.

Viva Questions

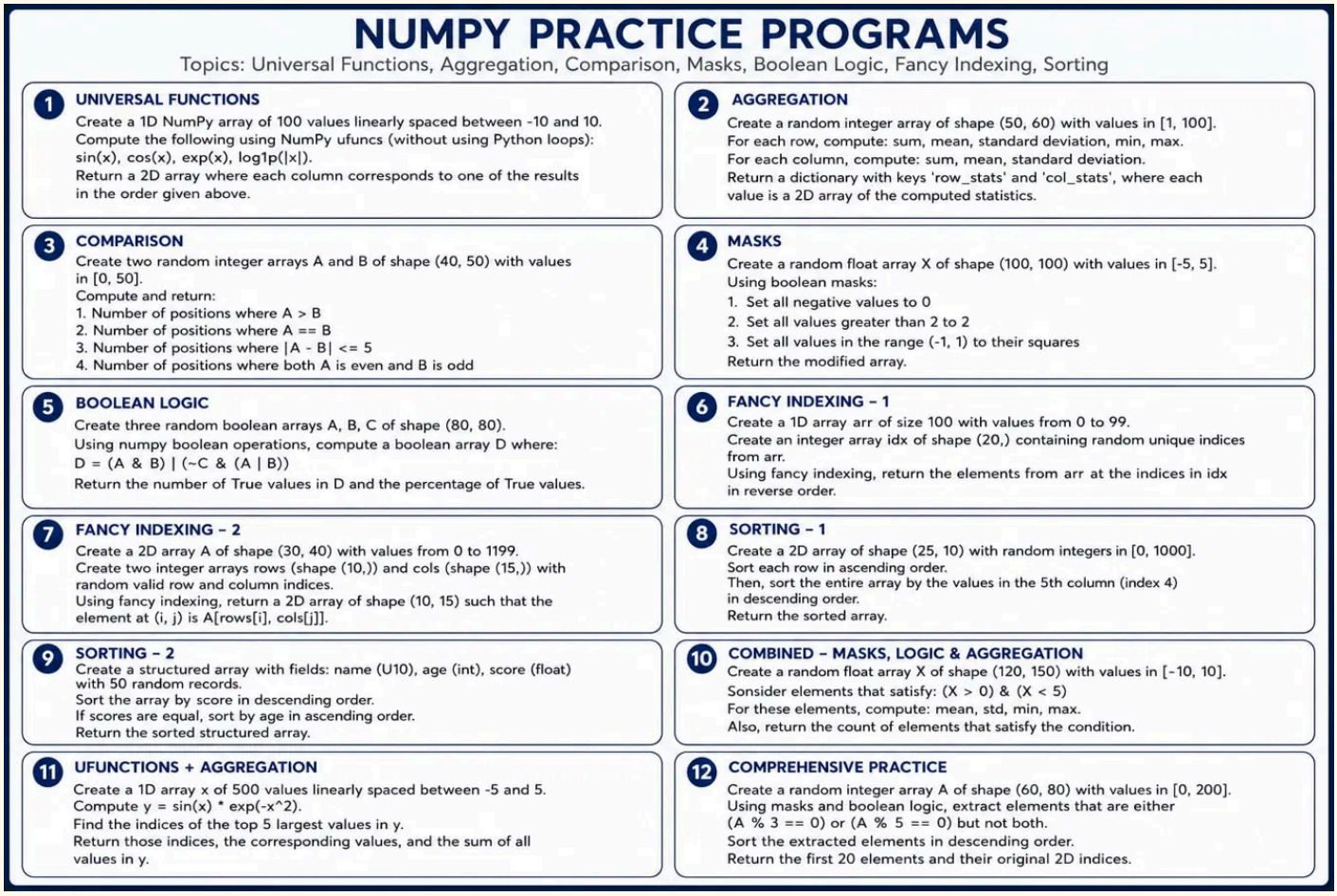
Q1. What is meant by building all combinations of two NumPy arrays?

Answer: It means pairing every element of the first array with every element of the second array.

Q2. Which function is used to change the data type of a NumPy array?
Answer: `astype()`.

Q3. Does `astype()` modify the original array?
Answer: No. It returns a new array with the changed data type.

Q4. Name some data types that can be used with `astype()`.
Answer: `int`, `float`, `str`, and `bool`.



NUMPY PRACTICE PROGRAMS

Topics: Universal Functions, Aggregation, Comparison, Masks, Boolean Logic, Fancy Indexing, Sorting

1 UNIVERSAL FUNCTIONS

Create a 1D NumPy array of 100 values linearly spaced between -10 and 10.
Compute the following using NumPy ufuncs (without using Python loops):
sin(x), cos(x), exp(x), log1p(|x|).
Return a 2D array where each column corresponds to one of the results in the order given above.

2 AGGREGATION

Create a random integer array of shape (50, 60) with values in [1, 100].
For each row, compute: sum, mean, standard deviation, min, max.
For each column, compute: sum, mean, standard deviation.
Return a dictionary with keys 'row_stats' and 'col_stats', where each value is a 2D array of the computed statistics.

3 COMPARISON

Create two random integer arrays A and B of shape (40, 50) with values in [0, 50].
Compute and return:
1. Number of positions where A > B
2. Number of positions where A == B
3. Number of positions where |A - B| <= 5
4. Number of positions where both A is even and B is odd

4 MASKS

Create a random float array X of shape (100, 100) with values in [-5, 5].
Using boolean masks:
1. Set all negative values to 0
2. Set all values greater than 2 to 2
3. Set all values in the range (-1, 1) to their squares
Return the modified array.

5 BOOLEAN LOGIC

Create three random boolean arrays A, B, C of shape (80, 80).
Using numpy boolean operations, compute a boolean array D where:
D = (A & B) | (~C & (A | B))
Return the number of True values in D and the percentage of True values.

6 FANCY INDEXING – 1

Create a 1D array arr of size 100 with values from 0 to 99.
Create an integer array idx of shape (20,) containing random unique indices from arr.
Using fancy indexing, return the elements from arr at the indices in idx in reverse order.

7 FANCY INDEXING – 2

Create a 2D array A of shape (30, 40) with values from 0 to 1199.
Create two integer arrays rows (shape (10,)) and cols (shape (15,)) with random valid row and column indices.
Using fancy indexing, return a 2D array of shape (10, 15) such that the element at (i, j) is A[rows[i], cols[j]].

8 SORTING – 1

Create a 2D array of shape (25, 10) with random integers in [0, 1000].
Sort each row in ascending order.
Then, sort the entire array by the values in the 5th column (index 4) in descending order.
Return the sorted array.

9 SORTING – 2

Create a structured array with fields: name (U10), age (int), score (float) with 50 random records.
Sort the array by score in descending order.
If scores are equal, sort by age in ascending order.
Return the sorted structured array.

10 COMBINED – MASKS, LOGIC & AGGREGATION

Create a random float array X of shape (120, 150) with values in [-10, 10].
Sonsider elements that satisfy: (X > 0) & (X < 5)
For these elements, compute: mean, std, min, max.
Also, return the count of elements that satisfy the condition.

11 UFUNCTIONS + AGGREGATION

Create a 1D array x of 500 values linearly spaced between -5 and 5.
Compute y = sin(x) * exp(-x^2).
Find the indices of the top 5 largest values in y.
Return those indices, the corresponding values, and the sum of all values in y.

12 COMPREHENSIVE PRACTICE

Create a random integer array A of shape (60, 80) with values in [0, 200].
Using masks and boolean logic, extract elements that are either (A % 3 == 0) or (A % 5 == 0) but not both.
Sort the extracted elements in descending order.
Return the first 20 elements and their original 2D indices.

```
import numpy as np

np.random.seed(42)  # For reproducibility

#
```

```
# 1. UNIVERSAL FUNCTIONS
#
_____
_____

print("=" * 60)
print("1. UNIVERSAL FUNCTIONS")
print("=" * 60)


x = np.linspace(-10, 10, 100)
result = np.column_stack([
    np.sin(x),
    np.cos(x),
    np.exp(x),
    np.log1p(np.abs(x))
])
print(f"Shape of result array: {result.shape}")
print(f"First 3 rows (sin, cos, exp, log1p|x|):\n{result[:3]}")
print(f"Last 3 rows:\n{result[-3:]}")
```

```
#
————————————————————————————————————————
————————

# 2. AGGREGATION

#
————————————————————————————————————————
————————

print("\n" + "=" * 60)

print("2. AGGREGATION")

print("=" * 60)


A = np.random.randint(1, 101, size=(50, 60))


row_stats = np.column_stack([

    A.sum(axis=1),

    A.mean(axis=1),
```

```
    A.std(axis=1),
    A.min(axis=1),
    A.max(axis=1)
])

col_stats = np.column_stack([
    A.sum(axis=0),
    A.mean(axis=0),
    A.std(axis=0),
    A.min(axis=0),
    A.max(axis=0)
])

stats = {'row_stats': row_stats, 'col_stats': col_stats}

print(f"Array shape: {A.shape}")
```

```
print(f"row_stats shape (50 rows × [sum,mean,std,min,max]): {stats['row_stats'].shape}")

print(f"col_stats shape (60 cols × [sum,mean,std,min,max]): {stats['col_stats'].shape}")

print(f"\nRow stats (first 3 rows) [sum, mean, std, min, max]:")

print(np.round(stats['row_stats'][:3], 2))

print(f"\nCol stats (first 3 cols) [sum, mean, std, min, max]:")

print(np.round(stats['col_stats'][:3], 2))



# ──────────────────────────────────────────────────────────────────────────

# 3. COMPARISON

# ──────────────────────────────────────────────────────────────────────────

print("\n" + "=" * 60)

print("3. COMPARISON")
```

```
print("=" * 60)


A = np.random.randint(0, 51, size=(40, 50))
B = np.random.randint(0, 51, size=(40, 50))


count_A_gt_B   = np.sum(A > B)
count_A_eq_B   = np.sum(A == B)
count_diff_le5 = np.sum(np.abs(A - B) <= 5)
count_A_even_B_odd = np.sum((A % 2 == 0) & (B % 2 != 0))


print(f"1. Positions where A > B          : {count_A_gt_B}")
print(f"2. Positions where A == B         : {count_A_eq_B}")
print(f"3. Positions where |A - B| <= 5    : {count_diff_le5}")
print(f"4. Positions where A even, B odd   : {count_A_even_B_odd}")
```

```
#
_____
_____

# 4. MASKS
#
_____
_____

print("\n" + "=" * 60)
print("4. MASKS")
print("=" * 60)

X = np.random.uniform(-5, 5, size=(100, 100))
print(f"Original stats → min: {X.min():.3f}, max: {X.max():.3f}, mean: {X.mean():.3f}")

X[X < 0] = 0                    # 1. negatives → 0
X[X > 2] = 2                    # 2. > 2 → 2
mask_mid = (X > -1) & (X < 1)          # 3. (-1,1) → square
```

```
X[mask_mid] = X[mask_mid] ** 2


print(f"Modified stats → min: {X.min():.3f}, max: {X.max():.3f},
mean: {X.mean():.3f}")

print(f"Unique values sample (first 10 unique):
{np.unique(X.ravel())[:10].round(4)}")



#
------------------------------------------------------------
------------

# 5. BOOLEAN LOGIC

#
------------------------------------------------------------
------------

print("\n" + "=" * 60)

print("5. BOOLEAN LOGIC")

print("=" * 60)
```

```
A = np.random.randint(0, 2, size=(80, 80), dtype=bool)
B = np.random.randint(0, 2, size=(80, 80), dtype=bool)
C = np.random.randint(0, 2, size=(80, 80), dtype=bool)

D = (A & B) | (~C & (A | B))

true_count = np.sum(D)
true_pct   = true_count / D.size * 100

print(f"D shape: {D.shape}")
print(f"Number of True values : {true_count}")
print(f"Percentage of True    : {true_pct:.2f}%")


#
_____
_______

# 6. FANCY INDEXING – 1
```

```
#
_____
_____

print("\n" + "=" * 60)
print("6. FANCY INDEXING – 1")
print("=" * 60)

arr = np.arange(100)
idx = np.random.choice(100, size=20, replace=False)
result = arr[idx[::-1]]

print(f"arr (0..99), shape: {arr.shape}")
print(f"idx (20 unique random indices): {idx}")
print(f"idx reversed                  : {idx[::-1]}")
print(f"Elements at reversed indices  : {result}")
```

```
#
-------------------------------------------------------------
-----------

# 7. FANCY INDEXING – 2

#
-------------------------------------------------------------
-----------

print("\n" + "=" * 60)

print("7. FANCY INDEXING – 2")

print("=" * 60)


A = np.arange(1200).reshape(30, 40)

rows = np.random.randint(0, 30, size=10)

cols = np.random.randint(0, 40, size=15)


result = A[np.ix_(rows, cols)]


print(f"A shape: {A.shape}, values 0–1199")
```

```
print(f"rows indices (10): {rows}")

print(f"cols indices (15): {cols}")

print(f"Fancy-indexed result shape: {result.shape}")

print(f"Result (first 3 rows shown):\n{result[:3]}")



#
————————————————————————————————————————————————————————

# 8. SORTING – 1

#
————————————————————————————————————————————————————————

print("\n" + "=" * 60)

print("8. SORTING – 1")

print("=" * 60)


A = np.random.randint(0, 1001, size=(25, 10))
```

```
# Sort each row ascending
A_row_sorted = np.sort(A, axis=1)

# Sort entire array by 5th column (index 4) descending
sort_idx = np.argsort(A_row_sorted[:, 4])[::-1]
A_sorted = A_row_sorted[sort_idx]

print(f"Original shape: {A.shape}")
print(f"After row-sort + col-4 desc sort shape: {A_sorted.shape}")
print(f"First 5 rows of sorted array:\n{A_sorted[:5]}")
print(f"5th column (index 4) values (should be descending): {A_sorted[:, 4]}")
```

```
#
-------------------------------------------------------------
----------

# 9. SORTING – 2

#
-------------------------------------------------------------
----------

print("\n" + "=" * 60)

print("9. SORTING – 2 (Structured Array)")

print("=" * 60)


dt = np.dtype([('name', 'U10'), ('age', int), ('score', float)])

data = np.zeros(50, dtype=dt)

data['name']  = [f"P{i:02d}" for i in range(50)]

data['age']   = np.random.randint(18, 60, size=50)

data['score'] = np.random.uniform(0, 100, size=50).round(2)
```

```
# Sort: primary = score descending, secondary = age ascending
# Use lexsort (last key is primary)
sort_idx = np.lexsort((data['age'], -data['score']))
sorted_data = data[sort_idx]

print(f"Structured array, 50 records")
print(f"{'Name':<10} {'Age':>5} {'Score':>8}")
print("-" * 26)
for row in sorted_data[:10]:
    print(f"{row['name']:<10} {row['age']:>5} {row['score']:>8.2f}")
print("  ... (showing top 10 of 50)")


#
```

```
# 10. COMBINED – MASKS, LOGIC & AGGREGATION
#
————————————————————————————————————————
————————

print("\n" + "=" * 60)
print("10. COMBINED – MASKS, LOGIC &
AGGREGATION")
print("=" * 60)

X = np.random.uniform(-10, 10, size=(120, 150))
mask = (X > 0) & (X < 5)
selected = X[mask]

print(f"X shape: {X.shape}, values in [-10, 10]")
print(f"Condition: (X > 0) & (X < 5)")
print(f"Count of matching elements : {len(selected)}")
print(f"Mean  : {selected.mean():.4f}")
print(f"Std   : {selected.std():.4f}")
```

```
print(f"Min   : {selected.min():.4f}")
print(f"Max   : {selected.max():.4f}")



#
──────────────────────────────────────────────────────────
────────

# 11. UFUNCTIONS + AGGREGATION
#
──────────────────────────────────────────────────────────
────────

print("\n" + "=" * 60)
print("11. UFUNCTIONS + AGGREGATION")
print("=" * 60)

x = np.linspace(-5, 5, 500)
y = np.sin(x) * np.exp(-x**2)
```

```
top5_idx    = np.argsort(y)[-5:][::-1]   # indices of top-5 largest
top5_vals   = y[top5_idx]
total_sum   = y.sum()


print(f"x: 500 values in [-5, 5]")
print(f"y = sin(x) * exp(-x²)")
print(f"\nTop 5 largest y-value indices: {top5_idx}")
print(f"Corresponding y values       : {np.round(top5_vals, 6)}")
print(f"Sum of all y values          : {total_sum:.6f}")



#
————————————————————————————————————————————————————————————

# 12. COMPREHENSIVE PRACTICE
```

```
#
_____
_____

print("\n" + "=" * 60)
print("12. COMPREHENSIVE PRACTICE")
print("=" * 60)

A = np.random.randint(0, 201, size=(60, 80))

# Elements divisible by 3 OR 5, but NOT BOTH (XOR)
div3 = (A % 3 == 0)
div5 = (A % 5 == 0)
mask = div3 ^ div5                  # XOR = one but not both

extracted = A[mask]                 # 1D flattened values
indices_2d = np.argwhere(mask)      # original 2D indices

# Sort extracted values descending
```

```
sort_order = np.argsort(extracted)[::-1]
sorted_vals    = extracted[sort_order]
sorted_indices = indices_2d[sort_order]

top20_vals    = sorted_vals[:20]
top20_indices = sorted_indices[:20]

print(f"A shape: {A.shape}, values in [0, 200]")
print(f"Total elements matching (div by 3 XOR 5): {len(extracted)}")
print(f"\nTop 20 elements (descending) and their 2D indices:")
print(f"{'Rank':<6} {'Value':>6}  {'2D Index'}")
print("-" * 30)
for rank, (val, idx) in enumerate(zip(top20_vals, top20_indices), 1):
    print(f"{rank:<6} {val:>6}  ({int(idx[0])}, {int(idx[1])})")
```
